



COMP9321:

Data services engineering

Semester 1, 2018

Week 8 OMM ... Obtaining, Messaging, Managing Data

By John Shepherd, CSE UNSW

OMM ... the Zen of Data

Obtaining

- Extracting data from a variety of sources

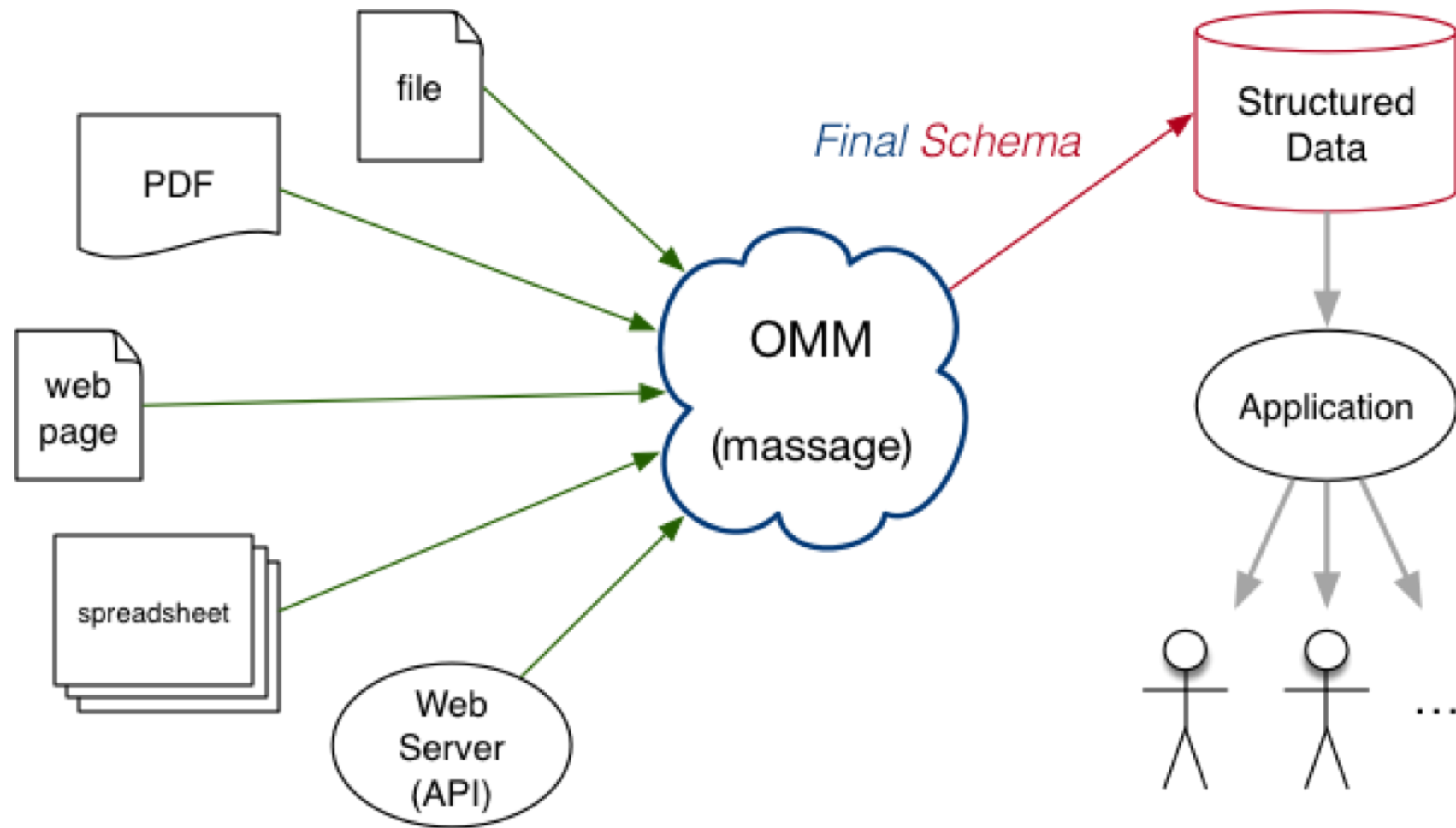
Massaging

- Transforming data into manageable form

Managing

- Loading the data into a platform for use

OMM ... Overview



Obtaining Data

Useful data can be found in many places

- on the Web, possibly via an API
- in documents in a file system
- in spreadsheets
- in videos
- etc. etc. etc.

and in a variety of formats

If it's already in a database, your job is done.

Obtaining Data (cont)

Formats for potentially useful information

- Unformatted text (in files)
- PDF documents (in files)
- HTML documents (web pages)
- XML documents (via web APIs)
- JSON data (often via web APIs)
- CSV data files (spreadsheets)

Unformatted Text Data

Unformatted text is generally *unfriendly*

Hi John,

Here are some stats on enrolments in 18s1.

Up by 20% on last year for undergraduates

(so there are 450 of them in COMP1511).

And postgrad coursework is up by 40% !!

510 enrolments in COMP9021.

Unformatted Text Data

The same info can be conveyed in many ways

Hey jas,

COMP1511 has 450 cse majors (20% increase).

And COMP9021 is a big jump (40%) and has

510 new PGCs.

Ouch!

Unformatted Text Data (cont)

Some text has informal structure

New enrolment figures for 18s1

- * PG: COMP9021, 510 enrolments
- * UG: COMP1511, 450 enrolments

But can we recognise and exploit such structure?

Unformatted Text Data (cont)

Requires sophisticated techniques like

- Natural language processing (NLP) (syntax)
- Machine Learning (ML) (patterns)

Such techniques are, at best, approximate

Specific problems ...

- Named entity recognition (who/what is COMP9021?)
- Training requirements (for ML)

“Unformatted” Text Data (cont)

Email metadata is more tractable (tags, dates, addresses, ...)

Delivered-To: j.a.shepherd@gmail.com

Received: by 10.80.194.65 with HTTP;

Tue, 24 Apr 2018 03:27:58 -0700 (PDT)

From: Helen Paik <h.paik@unsw.edu.au>

Date: Tue, 24 Apr 2018 20:27:58 +1000

Message-ID: <vW5eP7ZbLwKapvtGDQ@mail.gmail.com>

Subject: Lecture time/location

To: John Shepherd <jas@cse.unsw.edu.au>

Python has packages for this e.g. emaildata

PDF Documents

PDF documents have some structure

- which specifies content and layout commands
- including both text and (mainly) binary data (e.g. OCR)
- but structure is not necessarily helpful for extraction

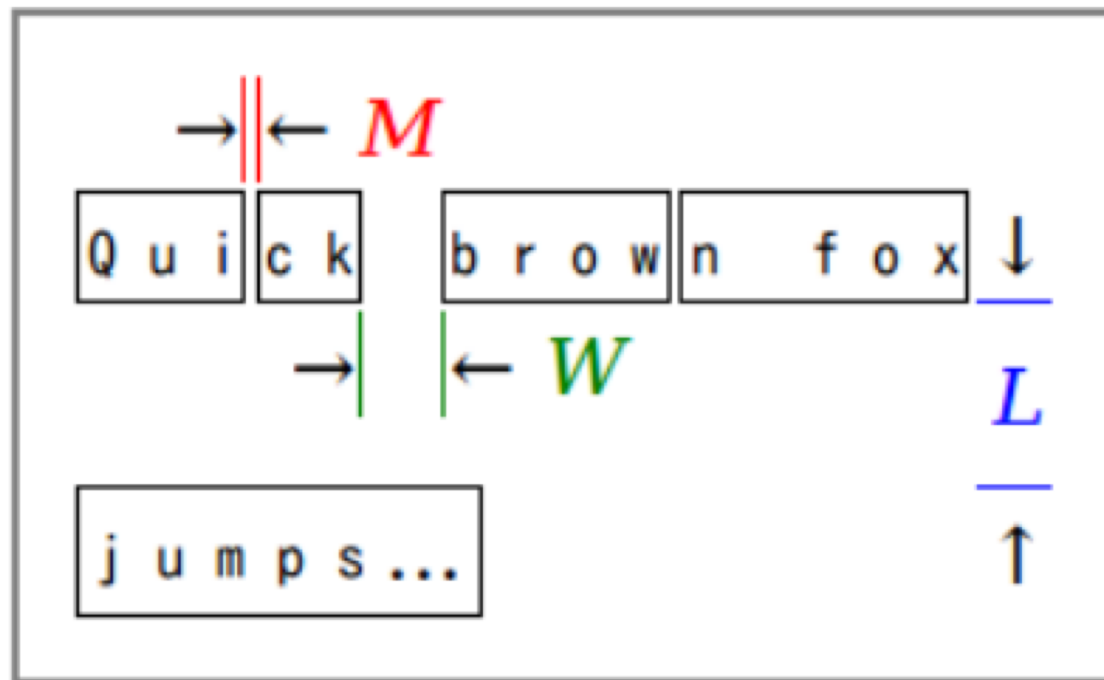
Python packages (e.g. pyPDF2) for dealing with PDF

- aim to extract the text from the document
- but don't aim to keep the layout structure

PDF Documents (cont)

Why even extracting text from PDFs is hard ...

- text chunks don't necessarily correspond to words



PDF Documents (cont)

More sophisticated extraction from PDF

PDFminer (including `pdf2txt.py` and `dumppdf.py`)

- toolset implemented in Python (v2)
- parses PDF documents, both text & images
- can convert PDFs to other formats (e.g. HTML)
- can output text + layout information

But still requires significant work to extract *data*

- requires NLP/ML, but aided by layout information

PDF Documents (cont)

Tables are often sources of useful data, but require ...

- finding table boundaries
- finding rows and columns
- finding cell boundaries
- extracting text from cells
- etc. etc.

Above is easy in HTML

Much harder in PDF

But e.g. pdftables.com

State	City	Town	POP
New York	Rensselaer	Troy	1
		Brunswick	2
	St. Lawrence	Potsdam	3
		Canton	4
California	San Diego	Coronado	5
		Del Mar	6
	Los Angeles	Malibu	7
		Compton	8

HTML Documents

HTML documents include explicit **markup**

- which is more *semantic* than PDF layout data
- making it easier to recognise document components
- but content is still semi-structured (“at creator’s whim”)

However, much HTML these days is generated

- giving a regular structure which is parseable

HTML Documents (cont)

Example HTML (i)

```
<h1>Enrolments</h1>  
<ul>  
<li> COMP9021 500 </li>  
<li> COMP1511 450 </li>  
</ul>
```

Easy to find structure; recognising columns harder.

HTML Documents (cont)

Example HTML (ii)

```
<h1>Enrolments</h1>
```

```
<table>
```

```
<tr> <th>Course</th> <th>Enrols</th> </tr>
```

```
<tr> <td>COMP9021</td> <td>500</td> </tr>
```

```
<tr> <td>COMP1511</td> <td>450</td> </tr>
```

```
</table>
```

Easy to find columns; headings assist with semantics.

HTML Documents (cont)

The Python BeautifulSoup library allows analysis of HTML

Example:

```
<!doctype html>
<html>
  <head><title> A simple page </title></head>
  <body>
    <p> Some simple content.</p>
  </body>
</html>
```

Assume above is in `http://xyz.com/p.html`

HTML Documents (cont)

BeautifulSoup Example (cont):

```
import requests
from bs import BeautifulSoup
page = requests.get("http://xyz.com/p.html")
# produce a nested structure representing the HTML
soup = BeautifulSoup(page.content, 'html.parser')
# get subcomponents of the html component
html = list(soup.children)[2]
...
```

HTML Documents (cont)

BeautifulSoup Example (cont):

...

```
# get the <body>...</body>
body = list(html.children)[3]
# get the <p>...</p> text
p = list(body.children)[1]
para = p.get_text()
```

Can use text and structure to explore document

HTML Documents (cont)

Combined with other Python modules e.g. regexps

- BeautifulSoup provides powerful tools to extract text
- can also place text in context within page structure
- can allow extraction of structured data
 - from within known page structures
 - or using specific patterns of tags/text
 - which is often the case nowadays
 - when HTML is mostly generated by scripts

HTML Documents (cont)

Example: UNSW Handbook

Easy to navigate and scrape because ...

- all program/stream/course pages have same structure
- significant amount of cross-linkage among pages
- index pages give links to all pages of given type

<http://www.handbook.unsw.edu.au/2018/index.html>

Example: Bojangles

Timetable generator
implemented by
CSE student
using data scraped
from [classutil](https://classutil.unsw.edu.au/)

	Monday	Tuesday	Wednesday	Thursday	Friday
9:00				COMP2521 Lecture	
10:00			COMP1531 Lecture		
11:00		COMP1531 TLB	COMP1531 Lecture		COMP2521 TLB
12:00	COMP2521 Lecture	COMP1531 TLB			COMP2521 TLB
13:00			COMP1521 TLB		
14:00		COMP1521 Lecture	COMP1521 TLB		COMP1531 Lecture
15:00					
16:00			COMP1521 Lecture		
17:00					

XML and JSON Data

XML and JSON are already structured data

- good for representing hierarchical structure
- have tags to indicate type of data (metadata)
- have much software to traverse their content

Tags help to massage into target structure

Will only look at XML ... JSON is similar (except Xpath)

XML and JSON Data (cont)

Example XML

```
<?xml version="1.0"?>
<countries>
  <country name="Liechtenstein">
    <rank>2</rank> <year>2008</year>
    <neighbour name="Austria" direction="E"/>
    <neighbour name="Switzerland" direction="W"/>
  </country>
  <country name="Singapore">...</country>
  <country name="Australia">...</country>
</countries>
```

XML and JSON Data (cont)

Example XML: `xml.etree.ElementTree`

```
import xml.etree.ElementTree as ET
tree = ET.parse('country_data.xml')
root = tree.getroot()
root.tag      # displays 'countries'
for c in root:
    print c.tag c.attrib
# displays ...
# country {'name': 'Liechtenstein'}
# country {'name': 'Singapore'}
# country {'name': 'Australia'}
```

XML and JSON Data (cont)

ElementTree also provides some support for Xpath

- extract data from XML trees by query

```
import xml.etree.ElementTree as ET
tree = ET.parse('country_data.xml')
# Nodes with name='Singapore' and a 'year' child
root.findall(".//year/..[@name='Singapore']")
# Neighbour nodes that are 2nd children
root.findall(".//neighbour[2]")
```

XML and JSON Data (cont)

Much useful data is available in JSON format

- typically from web service API's

Python provides JSON library module (`json`)

- `dump()` serializes Python objects as JSON data
- `load()` converts JSON data into Python objects
- apply standard Python methods to load'ed objects

CSV Data

Most CSV data is effectively a (relational) table

- however may not be normalised (in RDB sense)

Much CSV data is produced from spreadsheets

- column headings provide metadata (if available)

Example:

```
"ZID", "Name", "Degree", "WAM"  
"9300035", "Shepherd, John", "3778", "80.5"  
"9601234", "Paik, Helen", "8543", "95.1"
```

CSV Data (cont)

Python has several CSV modules

`csv` provides basic reading/writing of CSV data

- each row becomes a Python list
- collection of rows is a list of lists

`pandas` provides reading/writing CSV data

- also, an abstraction of the data (`DataFrame`)
- plus a range of operators for filtering/calculating

Obtaining Data

Much (all?) data can be accessed via the Web

Only collect data from **public** sources

- respect usage restrictions and copyright

If crawling and scraping, do it **slowly** (over weeks)

- check daily/frequency-of usage limits
- otherwise, you or cse will be blocked from site

Same applies to accessing data via web APIs

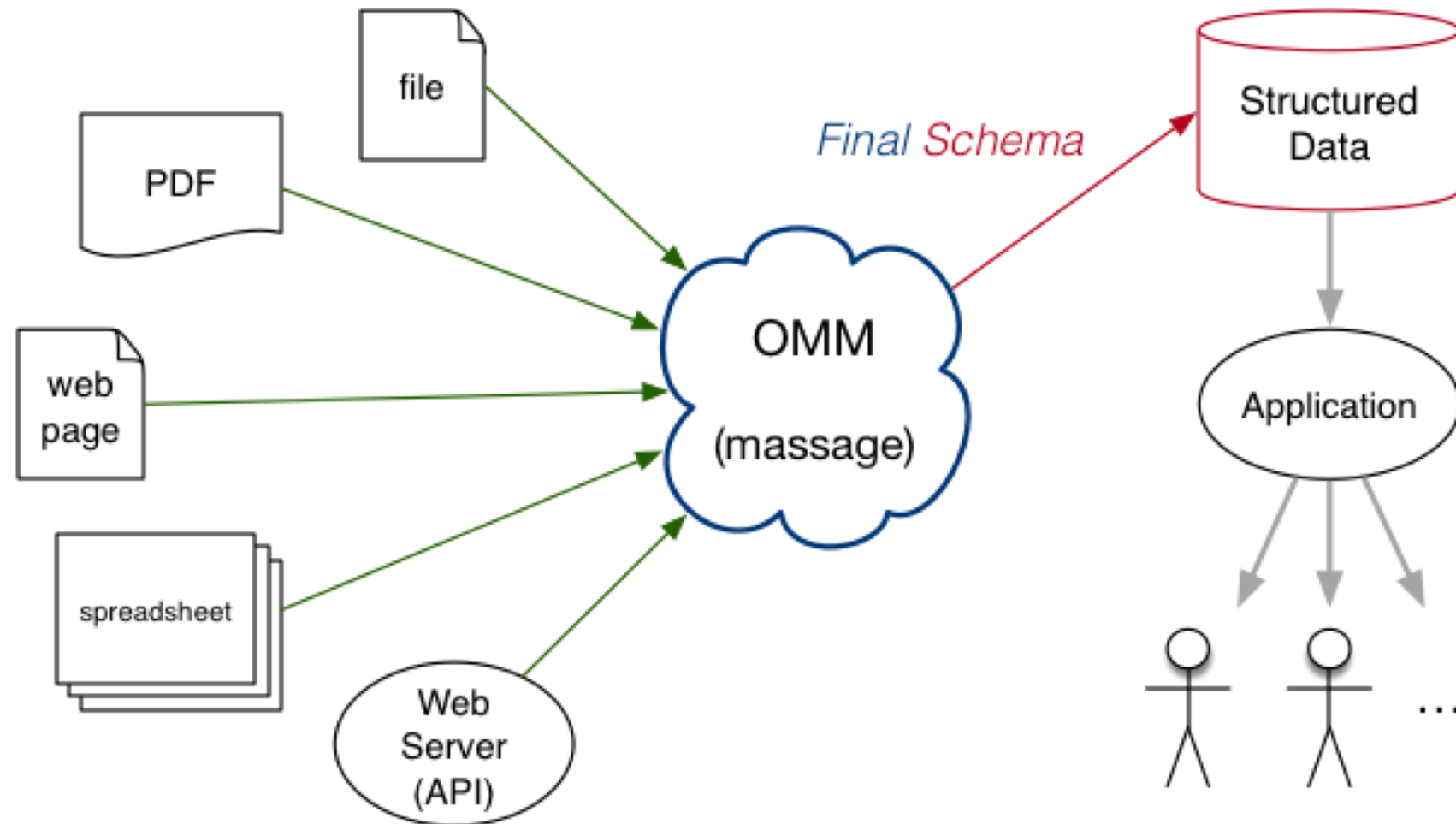
Obtaining Data (cont)

Ensure that any identifying data is de-identified

- any form of ID (e.g. zID) ... definitely (randomise)
- person names ...definitely (random name generator)
- phone numbers ... definitely
- organisation names ... maybe
- addresses ... definitely
- geo-locations ... maybe (google maps)

Tools exist for generating “realistic” data

The story so far ...



Massaging Data

Massaging data aims to

- take a bunch of **raw data** and metadata
- take a **schema** for storing this data
- then **map** the raw data into the schema

In other words ...

- populate the schema with the extracted data

Potential problem: may not have (enough) metadata

Massaging Data (cont)

Of the above systems and formats ...

- most allow you to extract raw text/numeric data items
- some allow you to extract metadata
- some allow you to determine relationships between data

But none directly provide a way of

- inserting data into *appropriate* slots in the **final schema**

Massaging Data (cont)

Potential problems:

- may not have (enough) metadata to match
- data may have a different schema
- some data values may be missing / incorrect

Example:

“ZID”, “Name”, “Degree”

“9300035”, “Shepherd, John”, “378”, “80.5”

“9601234”, “Paik, Helen”, “”, “95.1”

Massaging Data (cont)

If you have

- a final schema
- some pieces of raw data (no meta-data)

It is generally unclear

- what data item goes where (ambiguous)
- whether any data items are missing
- whether some data items are irrelevant
- which items belong to the same schema entity

Massaging Data (cont)

Example:

Data = 2312345, 3254321, 3707, 3978, 80, 75

Schema = (zID, Name, Program, WAM)

Could work out which columns data belongs in

- based on knowledge of underlying domains
- e.g. zIDs are 7-digits, program codes are 4-digits, etc.

In general, it is possible that domains might overlap, etc.

Massaging Data (cont)

Example:

Data = 2312345, 3254321, 3707, 3978, 80, 75

Schema = (zID, Name, Program, WAM)

How to work out which items belong together?

- 2312345 has 80 WAM or 75 WAM?
- 3254321 is studying 3707 or 3978?

Positional/relationship information would be needed

Massaging Data (cont)

If you have

- a final **schema**
- some **metadata** and pieces of raw data

then you have a **schema matching** problem

Schema matching: reconciling independent schemas

- known problem in database community
- not satisfactorily solved after 30 years

Massaging Data (cont)

Simple example of schema :

Student = (zID, name, address, program, WAM)

vs

Student = (zID, family, given, street, town, state, pcode, program)

address = "1 Smith St, Smithton, VIC 3333"

vs

street = "1 Smith St", town = "Smithton",
state = "VIC", postcode = 3333

Example: Student Data

Normalised schema for UNSW data is straightforward

People(zID, name, career, email, phone,...)

Programs(pid, code, title, UoC, ...)

Streams(sid, code, title, UoC, ...)

Courses(cid, code, term, title, UoC, description, ...)

Classes(clid, type, cid, code, location, day, starts, ends, ...)

CourseEnrolments(zID, cid, mark, grade)

ClassEnrolments(zID, clid) *StreamEnrolments(zID, sid)*

ProgramEnrolments(zID, pid, stage, standing, ...)

Example: Student Data (cont)

What we get in enrolment downloads

Students(zID, name, email, ...)

*Enrolments(zID, career, programCode, streamCode,
termID, courseID, stage, term, uoc, classID,
courseTitle, courseCode1, courseCode2, faculty, school)*

For double-degree students

- two entries for each enrolled course, one for each stream

Example: Student Data (cont)

Goal is to build Enrolments table

- need to build other tables “simultaneously”

Assume $S = \text{Students}$, $E = \text{Enrolments}$

Mapping:

$S.zID \rightarrow \text{People.zID}$ $S.name \rightarrow \text{People.name}$

$S.email \rightarrow \text{People.email}$ $E.career \rightarrow \text{People.career}$

$E.courseID \rightarrow \text{Courses.cid}$ $E.termID \rightarrow \text{Courses.term} \dots$

Example: Student Data (cont)

Goal is to build Enrolments table

Mapping (cont):

E.courseCode1+courseCode2 -> Courses.code

E.courseTitle -> Courses.title E.uoc -> Courses.uoc

E.zID -> CourseEnrolments.zID

E.courseID -> CourseEnrolments.cid

E.zID -> ClassEnrolments.zID

E.classID -> ClassEnrolments.clid

Example: Student Data

Normalised schema for UNSW data is straightforward

People(zID, name, career, email, *phone*, ...)

Programs(pid, *code*, title, UoC, ...)

Streams(sid, *code*, title, UoC, ...)

Courses(cid, code, term, title, UoC, *description*, ...)

Classes(*clid*, type, *cid*, code, location, day, starts, ends, ...)

CourseEnrolments(zID, cid, *mark*, *grade*)

ClassEnrolments(zID, clid) *StreamEnrolments*(zID, sid)

ProgramEnrolments(zID, pid, *stage*, standing, ...)

Managing Data

Once your data has been aligned with the schema

- transfer to whatever data management system implements your schema (e.g. RDBMS, noSQL, ...)
- data management is then handled by that system

Updates raise new problems

- assume that messaging still works as before
- but is the new data an update? or is it really new?
- e.g. (zID,course,mark) changes from 45 to 55 ... re-taken?

Summary

Data is available ubiquitously, in a variety of formats

Extracting usable data from most formats is challenging

- Need at least some metadata clues for non-trivial schemas

Fitting extracted data into a schema is also challenging

Challenging means there are only *approximate* solutions

- can't get 100% accuracy except in highly constrained scenarios

Managing extracted data can be handled by your DBMS